

Teoría 10: EXCEPCIONES

pueden ser atendidas por:

- mecanismos del lenguaje (construcciones especializadas, excepciones) (Nosotros nos concentramos en esta)
- mecanismos de hardware (interrupciones)
- mecanismos del Sistema Operativo (comunicación entre procesos (IPC), señales)

¿Qué es una excepción?

Condición inesperada o inusual, que ocurre durante la ejecución del programa y no puede ser manejada en el contexto local, es decir, necesito de una herramienta para manejar el error y poder continuar con la ejecución del código.

interrumpe el flujo normal de ejecución y ejecuta un controlador de excepciones registrado previamente.

Los lenguajes pueden proporcionar estas facilidades, pero no todos funcionan igual.

Controlador de excepciones/Manejador

Es una sección de código que se encarga de manejar una excepción en un programa.

Su objetivo principal es proporcionar una forma de recuperarse de un error o falla, permitiendo que el programa continúe ejecutándose en lugar de detenerse abruptamente.

Tipos de excepciones:

- Implícitas: Definidas por el lenguaje (built-in)
- Explícitas: Definidas por el programador

¿Qué se debe tener en cuenta un lenguaje que provee manejo de excepciones?

- ¿Cuáles son las excepciones que se pueden manejar? ¿Cómo se definen?
- ¿Cómo se maneja una excepción y cuál es su ámbito?
- ¿Cómo se lanza una excepción?

- ¿Cómo especificar los controladores de excepciones que se han de ejecutar cuando se alcanza las excepciones?
- ¿A dónde se cede el control cuando se termina de atender una excepción?
- ¿Cómo se propagan las excepciones?
- ¿Hay excepciones predefinidas?
- ¿Hay situaciones no controladas que lleven a mayores fallos?

Los lenguajes deben proveer instrucciones para:

- Definición de la excepción
- Levantamiento de una excepción
- Manejador de la excepción

Punto de retorno:

Después de atender a una excepción, el punto de retorno dependerá del flujo de ejecución del programa y de cómo se haya diseñado el manejo de excepciones en el código. También va a depender del lenguaje.

Se puede tener en cuenta:

- Continuar la ejecución normal del programa
- Retornar a un estado anterior
- Propagar la excepción
- Terminar la ejecución del programa

Continuar la ejecución normal del programa:

- Si después de manejar una excepción el programa puede continuar la ejecución del código restante sin problemas
- El punto de retorno será definido por el lenguaje (por ejemplo, el siguiente bloque de código después del bloque de manejo de excepciones o siguiente instrucción)

Retornar a un estado anterior:

- Cuando el manejo de excepciones puede requerir que el programa regrese a un estado anterior o deshaga acciones realizadas antes de que se produjera la excepción.

Propagar la excepción:

- En el controlador de excepciones no puede manejar completamente la excepción, puede optar por propagarla a un nivel superior en la jerarquía de llamadas.
- El punto de retorno sería el controlador de excepciones en el nivel superior que pueda manejar la excepción o decidir cómo manejarla.

Terminar la ejecución del programa:

- En situaciones excepcionales o críticas, es posible que el controlador de excepciones determine que no se puede continuar ejecutando el programa de manera segura.
- El punto de retorno puede ser la finalización del programa o alguna acción específica de cierre antes de la finalización.

2 MODELOS DE MANEJO DE EXCEPCIONES

-reasuncion

-terminacion

Reasunción:

se refiere a la posibilidad de retomar la ejecución normal del programa después de manejar una excepción. El controlador de excepciones realiza las acciones necesarias para manejar la excepción (medidas correctivas) y luego el programa continúa su ejecución a partir del punto donde se produjo la excepción.

- PL/1

Terminación:

El controlador de excepciones realiza las acciones necesarias para manejar la excepción, pero no se retorna al punto donde se produjo la excepción (invocador), continúa su ejecución a partir de la finalización del manejador. Es el más utilizado actualmente

- ADA

- CLU
- C++
- Java
- Python
- PHP

PL/1:

- Las excepciones se llaman CONDITIONS en PL/1
- Los manejadores se declaran con la sentencia ON: ON CONDITION(Nombre-excepción) Manejador
- El Manejador puede ser una instrucción o un bloque (entre beginy end)
- Las excepciones se lanzan explícitamente con la palabra clave SIGNAL: SIGNAL CONDITION(Nombre-excepción)
- Los manejadores se ligan dinámicamente con las excepciones. Una excepción siempre estará ligada con el último manejador definido. (Manejo de pila de manejadores de excepciones)

ADA

- Aplica criterio de Terminación

Cada vez que se produce una excepción, se termina el bloque dónde se levantó y se ejecuta el manejador asociado, y continúa luego.

Las excepciones se definen/declaran en la zona de definición de variables y tienen el mismo alcance en la unidad. Su formato para declarar es: MiExcepcion: exception;

- La lista de controladores de excepciones lleva el prefijo de la palabra clave exception.

- Cada controlador lleva el prefijo de la palabra clave when (con un formato específico), seguido de las acciones.
- Se puede utilizar when others para capturar cualquier excepción no especificada,
- debe colocarse al final del bloque de manejo de excepciones,
- posee efectos colaterales
- Las excepciones se levantan explícitamente con la palabra clave raise.
- Los manejadores pueden agregarse y encontrarse al final de diferentes unidades de programa: Subprograma, Bloque, Procedimiento, Paquete o Tarea que maneja la excepción.

Propagación

- Si la unidad que genera la excepción proporciona un manejador para la misma, el control se transfiere inmediatamente a ese manejador:
 - se omiten las acciones que siguen al punto en el que se generó la excepción,
 - se ejecuta el manejador
 - luego el programa continúa su ejecución normalmente, desde la instrucción que sigue al manejador.
- **Si la unidad no proporciona un manejador se busca por propagación**

Propagación

Si la unidad que genera la excepción no proporciona un manejador, se debe buscar ese manejador dinámicamente.

- Se termina la unidad (bloque, paquete, subprograma o tarea) dónde se produce la excepción.
- Si el manejador no se encuentra en ese lugar la excepción se propaga dinámicamente (quién lo llamó). Esto significa que se vuelve a levantar en otro ámbito.
- Siempre tener en cuenta el alcance, puede convertirse en anónima. Al propagarse a otras unidades la variable excepción declarada ya no está en el alcance y quedará sin nombre y entrará por when others

Si la variable de la excepción sale dinámicamente de su unidad ya que esta no la puede manejar, se vuelve anónima y pierde su alcance

Uso del Raise:

- La utilidad de raise es poder lanzar excepciones que pueden ser definidas por el programador.
- una excepción se puede levantar nuevamente colocando solo la palabra raise.
- Para qué? el manejador podría realizar algunas acciones de recuperación y luego utilizar raise para volver a lanzar la excepción y permitir que se propague más arriba en la jerarquía de manejo de excepciones.

Tener en cuenta :

- La asociación de la excepción con el manejador es determinístico (variable $\leftrightarrow$ manejador de esa variable). Asigno mismo nombre a ambos
- Si se deseara continuar ejecutando las instrucciones de un bloque donde se lanza una excepción, es preciso “crear un bloque más interno”. Se usa Declare para amar una unidad, como se muestra en el ejemplo y luego agregar instrucciones restantes abajo. (simular reasunción)

Herramienta propia de ada, DECLARE es un bloque de código que tiene manejadores de excepciones, y al final de este bloque hay código, por lo tanto si se produce una excepción el final de código se va a ejecutar igual.

C++

- Utiliza el criterio de Terminación.
- Cuenta con excepciones estandar predefinidas.
- Try para indicar los bloques donde pueden llegar a levantarse excepciones. Los mismos van precedidos por esta palabra clave Try
- Catch se utilizada para especificar los manejadores. Catch(NombreDeLaExcepción)
- las cláusulas catch deben estar después del bloque try y antes de cualquier código que esté fuera del bloque try.
- Los manejadores van asociados a bloques { }
- Throw Se utiliza para lanzar explícitamente una excepción

Se puede usar catch(int) //int o long o char etc

Funcionamiento:

- El bloque try que contiene código que puede lanzar una excepción.
- Si se lanza una excepción el control se transfiere inmediatamente a la cláusula catch.
- Si la excepción coincide con el tipo especificado en la cláusula catch, se ejecuta el bloque de código de esa cláusula catch.
- Si la excepción se maneja exitosamente, la ejecución continúa después del bloque try-catch.
- Si no se encuentra una catch correspondiente o no se maneja la excepción, la excepción puede propagarse hacia bloques try-catch externos.
- Sinó puede resultar en una finalización abrupta del programa.

Permite pasar parámetros al levantar la excepción: `throw MiExcepcion(parametro1, , parametroN);`

- permite que el programador especifique de manera precisa la intención de una rutina, al especificar tanto el comportamiento normal esperado (los datos que puede aceptar y devolver) como sus comportamientos anormales pasándole los parámetros al throw.

Ejemplo

```
void rutina ( ) throw (Ayuda, Zerodivide);
```

rutina se declara como una función void que no devuelve ningún valor y que puede lanzar 2 tipos de excepciones: Ayuda y Zerodivide.

- Si lanzó otra excepción que no está contemplada en el listado de la Interface. En este caso no se propaga la excepción y una función especial `unexpected()` se ejecuta automáticamente, que eventualmente causa `abort()` que provoca el final del programa.
- Si colocó en su interface el listado de posibles excepciones a alcanzar En este caso Si se propaga la excepción.

Si una excepción se propaga repetidamente y nunca se encuentra un manejador coincidente, llama automáticamente a una función especial llamada `terminate()`. `terminate()` puede ser redefinido al igual que `unexpected()` por el programador. Su comportamiento predeterminado, e ventualmente aborta la ejecución del programa

- Si no se proporciona ninguna lista throw Significa que CUALQUIER excepción puede ser propagada.
- Si se colocó en su interface una lista vacía throw() Significa que NINGUNA excepción será propagada.

EXCEPCIONES – CLU

- Utiliza el criterio de Terminación.
- Solamente pueden ser lanzadas por los procedimientos. si una instrucción genera una excepción, el procedimiento que contiene la instrucción retorna anormalmente al generar la excepción. Un procedimiento no puede manejar una excepción generada por su ejecución, quien llama al procedimiento debe encargarse de manejarla.
- La excepciones que un procedimiento puede lanzar se declaran en su encabezado.
- Se lanzan explícitamente con la palabra clave signal
- Los manejadores se colocan al lado de una sentencia simple o compleja y llevan la palabra clave when .

<instrucción> except <lista_de_controladores> end

donde <instrucción> puede ser cualquier instrucción (compuesta) del lenguaje. Si la ejecución de una invocación de procedimiento dentro de <instrucción> genera una excepción, el control se transfiere a <lista_de_controladores>.

- Se pueden pasar parámetros a los manejadores.
- Una excepción se puede volver a levantar una sola vez utilizando resignal
- Una excepción se puede levantar en cualquier lugar del código

Si no encuentra el manejador es por Propagación al producirse una excepción:

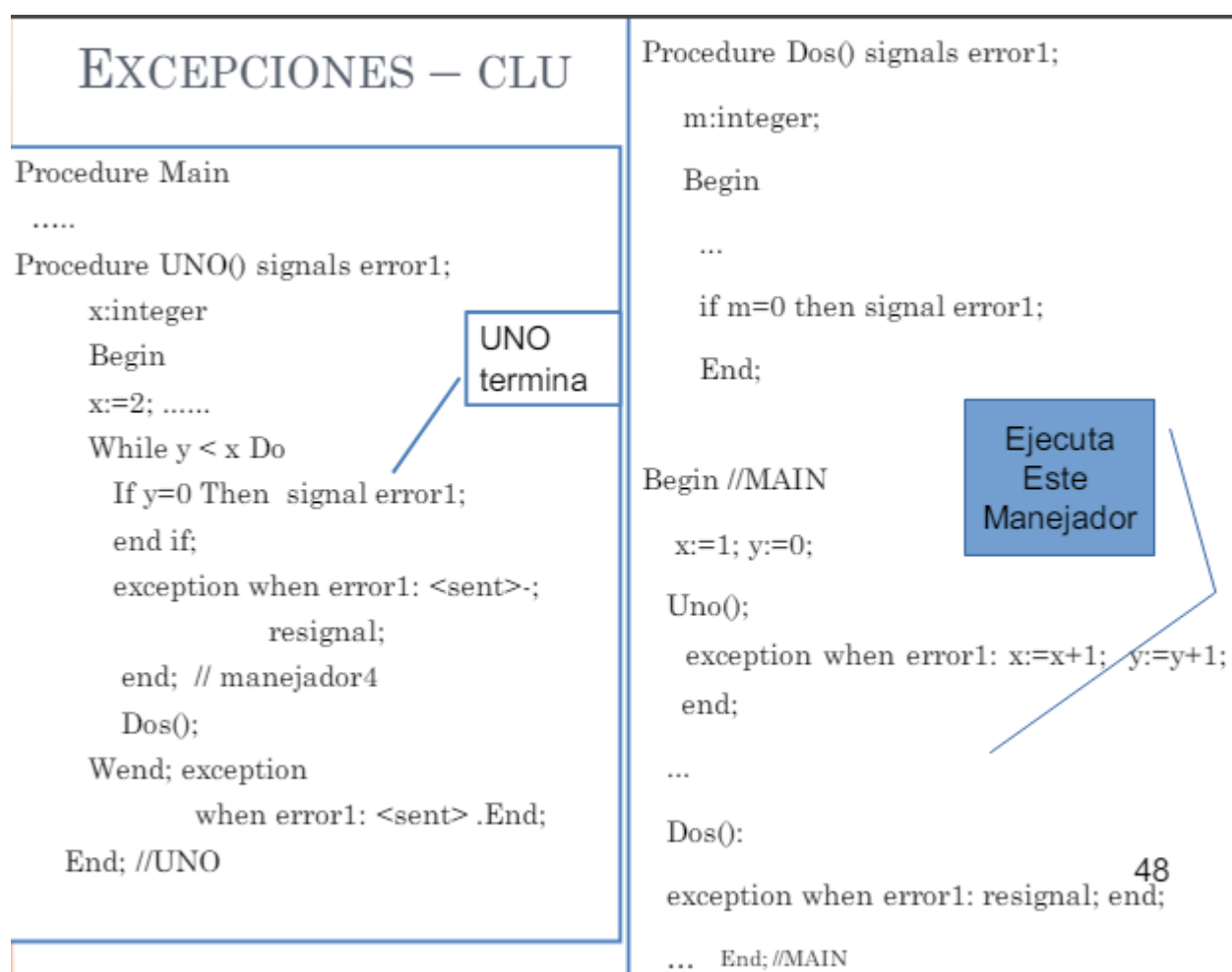
- Se termina el procedimiento donde se levantó la excepción y devuelve el control al llamante inmediato donde se debe encontrar el manejador.
- Si el manejador se encuentra en ese ámbito, se ejecuta y luego se pasa el control a la sentencia siguiente a la que está ligado dicho manejador.

- Si el manejador no se encuentra en ese lugar la excepción se propaga estáticamente en las sentencias asociadas. Esto significa que el proceso se repite para las sentencias incluidas estáticamente.
- En caso de no encontrar ningún manejador en el procedimiento que hizo la llamada se levanta una excepción failure y devuelve el control, terminando todo el programa

Cuando se tira una excepción la unidad termina y se busca el manejo de la excepción en la unidad que la llamo

Busco el manejador en la misma línea, sino busco estaticamente

Si no encuentra el manejo de la excepción en la unidad que lo llamo busca estáticamente. Seria quien contiene, ej el while



En este caso dos esta contenido en el while asi que ejecuta el manejador del while

JAVA

- Al igual que C++ las excepciones son objetos que pueden ser alcanzados y manejados por manejadores adicionados al bloque donde se produjo la excepción.
- Cada excepción está representada por una instancia de la clase Throwable o de una de sus subclases (Error y Exception)
- La gestión de excepciones se lleva a cabo mediante cinco palabras clave: try, catch, throw, throws, finally.
- Se debe especificar mediante la cláusula throws cualquier excepción que se envía desde un método.
- Se debe poner cualquier código que el programador desee que se ejecute siempre, en el método finally.

FASES DEL TRATAMIENTO DE EXCEPCIONES

- **Detectar e informar del error:**
 - Lanzamiento de Excepciones → throw
 - Un método detecta una condición anormal que le impide continuar con su ejecución y finaliza “lanzando” un objeto Excepción.
 - **Recoger el error y tratarlo:**
 - Captura de Excepciones → bloque try-catch
 - Un método recibe un objeto Excepción que le indica que otro método no ha terminado correctamente su ejecución y decide actuar en función del tipo de error.
-

PHYTON

Se manejan a través de bloques try except pero agrega la sentencia else

El código de la clausula else se ejecuta solo si no se levanta una excepcion

El código de la clausula finally se ejecuta siempre

La clausula optional solo se ejecuta si no se levanta una excepción en el try except

La declaración try funciona de la siguiente manera:

- Primero, se ejecuta el bloque try (el código entre las declaraciones try y except).
- Si no ocurre ninguna excepción, el bloque except se saltea y termina la ejecución de la declaración try.
- Si ocurre una excepción durante la ejecución del bloque try, el resto del bloque se saltea. Luego, si su tipo coincide con la excepción nombrada luego de la palabra reservada except, se ejecuta el bloque except, y la ejecución continúa luego de la declaración try.
- Si ocurre una excepción que no coincide con la excepción nombrada en el except, esta se pasa a declaraciones try de más afuera; si no se encuentra nada que la maneje, es una excepción no manejada, y la ejecución se frena con un mensaje de error

¿Qué sucede cuando una excepción no encuentra un manejador en su bloque “try except”?

- Busca estáticamente Analiza si ese try está contenido dentro de otro y si ese otro tiene un manejador para esa excepción. Sino...
 - Busca dinámicamente Analiza quién lo llamó y busca allí
 - Si no se encuentra un manejador, se corta el proceso y larga el mensaje standard de error
 - Levanta excepciones explícitamente con “raise”
-

PHP

- Modelo de Terminación
- Una excepción puede ser lanzada (thrown), y atrapada ("caught")
- El código esta dentro de un bloque try,
- Cada bloque try debe tener al menos un bloque catch correspondiente.
- Las excepciones pueden ser lanzadas (o relanzadas) dentro de un bloque catch.
- Se puede utilizar un bloque finally después de los bloques catch

EXCEPCIONES EN PHP

- El objeto lanzado debe ser una instancia de la clase Exception o de una subclase de Exception. Intentar lanzar un objeto que no lo es resultará en un Error Fatal de PHP.
- Cuando una excepción es lanzada, el código siguiente a la declaración no será ejecutado, y PHP intentará encontrar el primer bloque catch coincidente. Si una excepción no es capturada, se emitirá un

Error Fatal de PHP con un mensaje "Uncaught Exception ..." ("Excepción No Capturada"), a menos que se haya definido un gestor con `set_exception_handler()`.